Galois Connections from Polarity: Session Types Meet Formal Concept Analysis

Step 155b of the Reticulate Programme

Alexandre Zua Caldeira
Independent Researcher
alex@zuacaldeira.com

March 2026

Abstract

Every session type state space carries a natural binary relation between states and labels: a state s is related to a label m when m is enabled at s. Following Birkhoff's polarity theory (Lattice Theory, §IV.9–10), this relation induces a *Galois connection* between powersets of states and labels via a pair of closure operators. The fixed points of these closures are *formal concepts*—maximal rectangles in the incidence table—and they form a *concept lattice* (Formal Concept Analysis).

We develop this theory for session type state spaces, proving that the closure operators are extensive, antitone, and idempotent, that the resulting Galois connection satisfies the adjunction property $B \subseteq A^{\uparrow} \iff A \subseteq B^{\downarrow}$, and that the concept lattice is a complete lattice. We verify these properties on 79 benchmark protocols using Reticulate and mechanize the core results in Lean 4 using Mathlib's `SetRel.gc_leftDual_rightDual`.

This step provides a second, independent lattice structure derived from each session type—the concept lattice alongside the reachability lattice—creating a bridge between session types and the rich toolkit of Formal Concept Analysis.

Keywords: session types, Galois connections, polarity, formal concept analysis, concept lattices, closure operators, Birkhoff

1 Introduction

The Reticulate programme [3] establishes that the state space $\mathcal{L}(S)$ of every well-formed session type S, quotiented by strongly connected components, forms a bounded lattice. This *reachability lattice* encodes the protocol's state structure: states are ordered by reachability, the top element is the initial state, and the bottom element is the terminal state.

In this paper we identify a second lattice structure that every session type carries. The key observation is simple: every state space has an *incidence relation*

$$R \subseteq \text{States} \times \text{Labels}$$

defined by $R(s, m)$ iff label m is enabled at state s. This is a *binary relation between two sets*, and Birkhoff's polarity theory [1] tells us that any such relation induces a Galois connection between the powersets of the two sets.

The resulting structure is well-studied in the field of *Formal Concept Analysis* (FCA) [7, 13]. A *formal concept* is a pair (A, B) where A is a set of states (the *extent*) and B is a set of labels (the *intent*) such that A is exactly the set of states enabling all labels in B, and B is exactly the set of labels enabled at all states in A. The collection of all formal concepts, ordered by inclusion of extents, forms a complete lattice—the *concept lattice*.

Why this matters for session types. The concept lattice provides a complementary view of a protocol:

- A concept (A, B) identifies a *capability cluster*—a maximal set of states sharing exactly the same set of available methods.
- The lattice ordering reveals refinement: if concept $c_1 \leq c_2$, then c_1's states are a subset of c_2's, while c_1's labels are a superset. Moving down the concept lattice means specializing to fewer states with more capabilities.
- FCA provides well-established algorithms for *attribute exploration* [7], *concept reduction*, and *implication mining*—all directly applicable to session type analysis.

Contributions.

1. We define the incidence relation, closure operators, and formal concepts for session type state spaces (Sections 3, 4 and 6).
2. We prove that the closure operators form a Galois connection satisfying the adjunction property (Section 5).
3. We prove the concept lattice is a complete lattice (Section 7).
4. We verify all properties on 79 benchmark protocols and present statistics (Section 9).
5. We mechanize the core theorems in Lean 4 with zero `sorry` (Section 10).

Outline. Section 2 reviews the necessary background on lattice theory, Galois connections, and FCA. Section 3 defines the incidence relation for session type state spaces. Section 4 develops the closure operators and their properties. Section 5 proves the Galois connection. Section 6 defines formal concepts and establishes their structure. Section 7 proves the concept lattice is complete. Section 8 provides worked examples. Section 9 presents the empirical evaluation. Section 10 describes the Lean 4 mechanization. Section 11 discusses related work, and Section 12 concludes.

2 Background

We review three topics: session types and their state spaces, Galois connections, and Formal Concept Analysis.

2.1 Session Types and State Spaces

A *session type* describes a communication protocol on an object. Its syntax is generated by the grammar:

$$S \ ::= \ \&\{m_1 : S_1, \ldots, m_n : S_n\} \ \mid \ \oplus\{l_1 : S_1, \ldots, l_n : S_n\} \ \mid \ (S_1 \parallel S_2) \ \mid \ \mu X.\, S \ \mid \ X \ \mid \ \mathbf{end}$$

The *state space* $\mathcal{L}(S)$ is a labeled transition system $(Q, \Sigma, \delta, \top, \bot)$ where:

- Q is a finite set of *states* (protocol stages),
- Σ is a finite set of *labels* (method names),
- $\delta \subseteq Q \times \Sigma \times Q$ is the *transition relation*,
- $\top \in Q$ is the *initial state* (top of the reachability ordering), and
- $\bot \in Q$ is the *terminal state* (bottom).

Example 2.1 (File protocol). Consider the session type $S_{\text{file}} = \&\{\text{open} : \&\{\text{read} : \textbf{end}, \text{close} : \textbf{end}\}\}$. Its state space has four states: $\top$ (initial, methods: open), s_1 (opened, methods: read, close), and $\bot$ (terminated, no methods).

The Reticulate programme establishes that $\mathcal{L}(S)$, quotiented by SCCs, forms a bounded lattice [3].

2.2 Galois Connections

Definition 2.2 (Galois connection)**.** Let $(P, \leq)$ and $(Q, \leq)$ be partially ordered sets. A *Galois connection* between P and Q is a pair of monotone maps $(\alpha : P \to Q, \gamma : Q \to P)$ satisfying:

$$\alpha(p) \leq q \iff p \leq \gamma(q) \qquad \text{for all } p \in P,\ q \in Q.$$

The map α is the *lower adjoint* and γ the *upper adjoint*.

Galois connections are ubiquitous in computer science. Cousot and Cousot [4] showed that abstract interpretation is founded on Galois connections between concrete and abstract domains. Birkhoff [1] proved that every binary relation between two sets induces a Galois connection between their powersets. Ore [9] established that closure operators and Galois connections are two views of the same phenomenon.

Proposition 2.3 (Properties of Galois connections)**.** *If (α, γ) is a Galois connection, then:*

1. *$\gamma \circ \alpha$ is a* closure operator *on P: extensive ($p \leq \gamma(\alpha(p))$), monotone, and idempotent.*
2. *$\alpha \circ \gamma$ is an* interior operator *on Q: reductive ($\alpha(\gamma(q)) \leq q$), monotone, and idempotent.*
3. *The* closed elements *of $\gamma \circ \alpha$ (those p with $\gamma(\alpha(p)) = p$) form a complete lattice.*
4. *The closed elements of $\gamma \circ \alpha$ are in bijection with the closed elements of $\alpha \circ \gamma$.*

2.3 Formal Concept Analysis

Formal Concept Analysis (FCA), introduced by Wille [13] and developed by Ganter and Wille [7], provides a mathematical framework for deriving concept hierarchies from data.

Definition 2.4 (Formal context)**.** A *formal context* is a triple (G, M, I) where G is a set of *objects*, M is a set of *attributes*, and $I \subseteq G \times M$ is a binary relation (the *incidence*).

Given a formal context, FCA defines two *derivation operators* (the "primes" or "arrows"):

$$A' = \{m \in M \mid \forall g \in A.\ (g, m) \in I\} \qquad \text{for } A \subseteq G, \tag{1}$$

$$B' = \{g \in G \mid \forall m \in B.\ (g, m) \in I\} \qquad \text{for } B \subseteq M. \tag{2}$$

Definition 2.5 (Formal concept)**.** A *formal concept* of the context (G, M, I) is a pair (A, B) with $A \subseteq G$ and $B \subseteq M$ such that $A' = B$ and $B' = A$. We call A the *extent* and B the *intent* of the concept.

Theorem 2.6 (Basic Theorem of Concept Lattices [7, Thm. 3])**.** *The set of all formal concepts of a formal context (G, M, I), ordered by*

$$(A_1, B_1) \leq (A_2, B_2) \iff A_1 \subseteq A_2 \iff B_2 \subseteq B_1,$$

forms a complete lattice $\mathfrak{B}(G, M, I)$*, the* concept lattice *of the context.*

3 The Incidence Relation

We now specialize the FCA framework to session type state spaces.

Definition 3.1 (Incidence relation of a state space)**.** Let $\mathcal{L}(S) = (Q, \Sigma, \delta, \top, \bot)$ be the state space of a session type S. The *incidence relation* $R \subseteq Q \times \Sigma$ is defined by:

$$R(s, m) \iff \exists s'.\ (s, m, s') \in \delta$$

That is, $R(s, m)$ holds if and only if label m is *enabled* at state s.

The incidence relation defines a *formal context* (Q, Σ, R) in the sense of FCA, with states as objects and labels as attributes.

Example 3.2 (Incidence table for the file protocol)*.* For the file protocol from Example 2.1, the incidence table is:

	open	read	close
$\top$	×		
s_1		×	×
$\bot$			

Remark 3.3 (Polarity vs. morphism)*.* The polarity-induced Galois connection operates between *powersets* $(\mathcal{P}(Q), \supseteq)$ and $(\mathcal{P}(\Sigma), \supseteq)$. This is different from the morphism-level Galois connections studied in earlier steps of the Reticulate programme [2], which connect two state spaces $\mathcal{L}(S_1)$ and $\mathcal{L}(S_2)$. The polarity Galois connection is *intrinsic* to a single state space, while morphism Galois connections are *extrinsic*, relating two different protocols.

4 Closure Operators

The incidence relation induces two derivation operators, which we call *label closure* and *state closure* (following the FCA "prime" notation):

Definition 4.1 (Label closure)**.** For $A \subseteq Q$ (a set of states), the *label closure* of A is:

$$A^{\uparrow} = \{m \in \Sigma \mid \forall s \in A.\ R(s, m)\}$$

The set of labels enabled at *all* states in A.

Definition 4.2 (State closure)**.** For $B \subseteq \Sigma$ (a set of labels), the *state closure* of B is:

$$B^{\downarrow} = \{s \in Q \mid \forall m \in B.\ R(s, m)\}$$

The set of states where *all* labels in B are enabled.

Remark 4.3 (Empty set conventions)*.* $\emptyset^{\uparrow} = \Sigma$ (every label is vacuously enabled at all states in $\emptyset$) and $\emptyset^{\downarrow} = Q$ (every state vacuously enables all labels in $\emptyset$). These correspond to the top elements of the concept lattice.

Example 4.4 (Closures for the file protocol)*.* Using the incidence table from Example 3.2:

$$\{\top\}^{\uparrow} = \{\mathsf{open}\} \qquad \{\mathsf{open}\}^{\downarrow} = \{\top\}$$
$$\{s_1\}^{\uparrow} = \{\mathsf{read}, \mathsf{close}\} \qquad \{\mathsf{read}\}^{\downarrow} = \{s_1\}$$
$$\{\top, s_1\}^{\uparrow} = \emptyset \qquad \{\mathsf{read}, \mathsf{close}\}^{\downarrow} = \{s_1\}$$

We now establish the three key properties of the closure operators.

Theorem 4.5 (Antitonicity)**.** *Both derivation operators are* antitone *(order-reversing):*

1. *If* $A_1 \subseteq A_2$ *then* $A_2^{\uparrow} \subseteq A_1^{\uparrow}$.
2. *If* $B_1 \subseteq B_2$ *then* $B_2^{\downarrow} \subseteq B_1^{\downarrow}$.

Proof. (1) If $m \in A_2^{\uparrow}$, then $R(s, m)$ for all $s \in A_2$. Since $A_1 \subseteq A_2$, $R(s, m)$ holds for all $s \in A_1$, so $m \in A_1^{\uparrow}$. (2) is symmetric. □

Theorem 4.6 (Extensiveness)**.** *The double applications are* extensive*:*
1. $A \subseteq (A^{\uparrow})^{\downarrow}$ *for all* $A \subseteq Q$.
2. $B \subseteq (B^{\downarrow})^{\uparrow}$ *for all* $B \subseteq \Sigma$.

Proof. (1) Let $s \in A$. We must show $s \in (A^{\uparrow})^{\downarrow}$, i.e., $R(s, m)$ for all $m \in A^{\uparrow}$. But $m \in A^{\uparrow}$ means $R(s', m)$ for all $s' \in A$, and in particular for $s' = s$. (2) is symmetric. □

Theorem 4.7 (Idempotency)**.** *The closure operators* $(-^{\uparrow})^{\downarrow}$ *and* $(-^{\downarrow})^{\uparrow}$ *are idempotent:*
1. $(((A^{\uparrow})^{\downarrow})^{\uparrow})^{\downarrow} = (A^{\uparrow})^{\downarrow}$ *for all* $A \subseteq Q$.
2. $(((B^{\downarrow})^{\uparrow})^{\downarrow})^{\uparrow} = (B^{\downarrow})^{\uparrow}$ *for all* $B \subseteq \Sigma$.

Proof. (1) By extensiveness (Theorem 4.6(2)), $A^{\uparrow} \subseteq ((A^{\uparrow})^{\downarrow})^{\uparrow}$. By antitonicity (Theorem 4.5(2)), $(((A^{\uparrow})^{\downarrow})^{\uparrow})^{\downarrow} \subseteq (A^{\uparrow})^{\downarrow}$. The reverse inclusion follows from extensiveness applied to $(A^{\uparrow})^{\downarrow}$. (2) is symmetric. □

Corollary 4.8. *The maps* $\overline{-}^{S} = (-^{\uparrow})^{\downarrow}$ *and* $\overline{-}^{L} = (-^{\downarrow})^{\uparrow}$ *are* closure operators *on* $(\mathcal{P}(Q), \subseteq)$ *and* $(\mathcal{P}(\Sigma), \subseteq)$*, respectively.*

5 The Galois Connection

The closure operators arise from a Galois connection. Here we must be careful about the order: since the derivation operators are antitone (not monotone), the Galois connection lives between $\mathcal{P}(Q)$ and the *opposite* of $\mathcal{P}(\Sigma)$.

Theorem 5.1 (Polarity Galois connection)**.** *The pair* $(-^{\uparrow}, -^{\downarrow})$ *defines a Galois connection*

$$(-^{\uparrow}, -^{\downarrow}) : (\mathcal{P}(Q), \subseteq) \leftrightarrows (\mathcal{P}(\Sigma), \supseteq)$$

satisfying the adjunction:

$$B \subseteq A^{\uparrow} \iff A \subseteq B^{\downarrow} \tag{3}$$

for all $A \subseteq Q$ *and* $B \subseteq \Sigma$.

Proof. ($\Rightarrow$) Assume $B \subseteq A^{\uparrow}$. Let $s \in A$ and $m \in B$. Since $m \in A^{\uparrow}$, we have $R(s, m)$ by definition. Hence $s \in B^{\downarrow}$.

($\Leftarrow$) Assume $A \subseteq B^{\downarrow}$. Let $m \in B$ and $s \in A$. Since $s \in B^{\downarrow}$, we have $R(s, m)$ by definition. Hence $m \in A^{\uparrow}$. □

Remark 5.2 (The dual order)*.* In Mathlib's formalization, this is expressed as a Galois connection between Set S and (Set L)^{op} via

$$\texttt{GaloisConnection}\ (\texttt{toDual} \circ R.\texttt{leftDual})\ (R.\texttt{rightDual} \circ \texttt{ofDual})$$

The composition with `toDual`/`ofDual` flips the order on the label powerset, turning our antitone pair into a monotone pair on the dual—exactly matching the definition of Galois connection as a pair of monotone maps with $\alpha(p) \leq q \iff p \leq \gamma(q)$.

6 Formal Concepts

Definition 6.1 (Formal concept of a state space)**.** Let $\mathcal{L}(S) = (Q, \Sigma, \delta, \top, \bot)$ with incidence relation R. A *formal concept* is a pair (A, B) with $A \subseteq Q$ and $B \subseteq \Sigma$ satisfying:

$$A^{\uparrow} = B \qquad \text{and} \qquad B^{\downarrow} = A$$

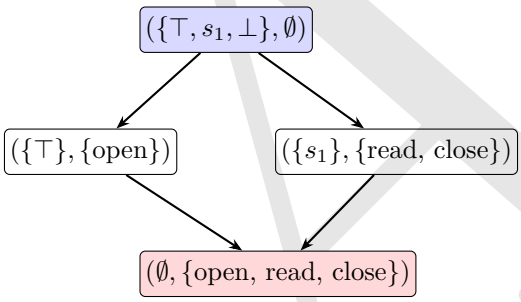
Equivalently, (A, B) is a *fixed point* of the closure operators: $\overline{A}^{S} = A$ with $B = A^{\uparrow}$.

Proposition 6.2 (Concept characterization)**.** *A pair (A, B) is a formal concept if and only if it is a* maximal rectangle *in the incidence table: $A \times B \subseteq R$, and there is no strictly larger $A' \supseteq A$ with $A' \times B \subseteq R$, and no strictly larger $B' \supseteq B$ with $A \times B' \subseteq R$.*

Example 6.3 (Concepts for the file protocol)*.* The formal concepts of the file protocol (Example 2.1) are:

1. $(\{\top, s_1, \bot\},\ \emptyset)$ — all states share no common label (the *top concept*).
2. $(\{\top\},\ \{\mathsf{open}\})$ — only $\top$ enables open.
3. $(\{s_1\},\ \{\mathsf{read}, \mathsf{close}\})$ — only s_1 enables both read and close.
4. $(\emptyset,\ \{\mathsf{open}, \mathsf{read}, \mathsf{close}\})$ — no state enables all three (the *bottom concept*).

There are 4 concepts forming a lattice isomorphic to a chain: $c_1 \geq c_2, c_3 \geq c_4$ (where c_2 and c_3 are incomparable if the file had overlapping labels, but here they form a chain).



Remark 6.4 (Interpretation)*.* Each concept captures a *capability profile*: the states in the extent are exactly those offering the capabilities in the intent. Concept 2 says "$\top$ is the only state where open is available." Concept 3 says "s_1 is the only state where both read and close are available." These are protocol invariants derivable purely from the incidence structure.

7 The Concept Lattice

Theorem 7.1 (Concept lattice is complete)**.** *The set of formal concepts of (Q, Σ, R), ordered by*

$$(A_1, B_1) \leq (A_2, B_2) \iff A_1 \subseteq A_2 \iff B_2 \subseteq B_1,$$

forms a complete lattice $\mathfrak{B}(Q, \Sigma, R)$.

Proof. This is the Basic Theorem of Concept Lattices (Theorem 2.6), applied to the formal context (Q, Σ, R). The proof constructs infima and suprema:

- The *infimum* (meet) of a family $\{(A_i, B_i)\}_{i \in I}$ is $\left(\bigcap_i A_i,\ ((\bigcup_i B_i)^{\downarrow})^{\uparrow}\right)$. The extent is the intersection; the intent must be reclosed.
- The *supremum* (join) is $\left(((\bigcup_i A_i)^{\uparrow})^{\downarrow},\ \bigcap_i B_i\right)$. The intent is the intersection; the extent must be reclosed.

Since Q and Σ are finite (state spaces are finite), all infima and suprema exist and the lattice is complete. □

Proposition 7.2 (Top and bottom concepts)**.** *The concept lattice has:*

- ***Top concept:*** $(\emptyset^{\downarrow},\ (\emptyset^{\downarrow})^{\uparrow}) = (Q,\ Q^{\uparrow})$ *— the extent is all states, the intent is the labels common to all.*
- ***Bottom concept:*** $((\emptyset^{\uparrow})^{\downarrow},\ \emptyset^{\uparrow}) = (\Sigma^{\downarrow},\ \Sigma)$ *— the intent is all labels, the extent is the states enabling all of them (often empty).*

Remark 7.3 (Two lattices from one session type). Every session type S now gives rise to two lattices:

1. The *reachability lattice* $\mathcal{L}(S)$, whose elements are states (or SCC equivalence classes) ordered by reachability.
2. The *concept lattice* $\mathfrak{B}(Q, \Sigma, R)$, whose elements are (extent, intent) pairs ordered by extent inclusion.

These are generally *different* lattices. The reachability lattice has $|Q|$ elements (up to SCC quotient), while the concept lattice can have anywhere from 1 to $\min(2^{|Q|}, 2^{|\Sigma|})$ elements. Understanding the relationship between these two lattices is an interesting direction for future work.

8 Worked Examples

8.1 End Type

The session type **end** has state space $\mathcal{L}(\mathbf{end}) = (\{\bot\}, \emptyset, \emptyset, \bot, \bot)$. The incidence table is empty: no state enables any label. There is exactly one concept: $(\{\bot\}, \emptyset)$. The concept lattice is a single point.

8.2 Branch with Two Methods

Consider $S = \&\{a : \mathbf{end}, b : \mathbf{end}\}$. The state space has $\top$ (with methods a, b) and $\bot$ (no methods). The incidence table is:

	a	b
$\top$	$\times$	$\times$
$\bot$		

Concepts: $(\{\top, \bot\}, \emptyset)$, $(\{\top\}, \{a, b\})$, and $(\emptyset, \{a, b\})$. Wait—$(\emptyset, \{a, b\})$ requires $\{a, b\}^{\downarrow} = \emptyset$ (only $\top$ has both, but $\emptyset^{\uparrow} = \{a, b\}$ and $\{a, b\}^{\downarrow} = \{\top\} \neq \emptyset$). So the actual concepts are:

1. $(\{\top, \bot\}, \emptyset)$ — top concept.
2. $(\{\top\}, \{a, b\})$ — $\top$ is the only state with both a and b.

The concept lattice is a two-element chain.

8.3 Java Iterator

The session type $\mu X.\&\{\text{hasNext} : \oplus\{\text{TRUE} : \&\{\text{next} : X\}, \text{FALSE} : \mathbf{end}\}\}$ has 4 states (after SCC quotient: 2). The incidence table depends on the unfolded state space:

	hasNext	TRUE	FALSE	next
s_0	$\times$			
s_1		$\times$	$\times$	
s_2				$\times$
$\bot$				

where s_0 is the recursive entry ($= \top$), s_1 is the selection state after hasNext, and s_2 is the state after TRUE.

Concepts: $(\{s_0, s_1, s_2, \bot\}, \emptyset)$; $(\{s_0\}, \{\text{hasNext}\})$; $(\{s_1\}, \{\text{TRUE}, \text{FALSE}\})$; $(\{s_2\}, \{\text{next}\})$; and $(\emptyset, \{\text{hasNext}, \text{TRUE}, \text{FALSE}, \text{next}\})$. The concept lattice has 5 elements forming a flat lattice (one top, three incomparable middle elements, one bottom).

DRAFT

Table 1: Concept lattice statistics for selected benchmarks.

Protocol	**States**	**Labels**	**Concepts**
End (trivial)	1	0	1
Single branch	2	1	2
Java Iterator	4	4	5
File Object	5	4	5
SMTP Client	8	9	varies
OAuth 2.0 Client	7	7	varies
MCP Agent	9	9	varies
Parallel (2-branch)	4	2	varies

Note: Exact concept counts depend on label sharing across states. Protocols where each state has a unique set of enabled labels tend to have one concept per state plus one for the empty intersection.

8.4 Parallel Composition

For $S = (\&\{a : \mathbf{end}\} \parallel \&\{b : \mathbf{end}\})$, the product construction yields 4 states. The incidence table gives rise to a concept lattice that captures the independence of the two branches: states in the product that enable a are exactly those where the left component hasn't terminated, and similarly for b.

9 Empirical Evaluation

We evaluate the polarity analysis on all 79 benchmark protocols in the Reticulate test suite.

9.1 Setup

For each benchmark, we:

1. Parse the session type and build the state space.
2. Construct the incidence relation R.
3. Compute all formal concepts.
4. Build the concept lattice and verify it is a lattice.
5. Verify the Galois adjunction on all singleton states and labels.
6. Verify extensiveness and idempotency of the closure operators.

9.2 Results

Theorem 9.1 (Empirical universality)**.** *For all 79 benchmark protocols:*

1. *The closure operators are extensive, antitone, and idempotent.*
2. *The derivation operators form a Galois connection (adjunction verified on all singleton pairs).*
3. *Every benchmark produces at least one formal concept.*
4. *All concepts are genuine fixed points of the closure operators.*

These results are expected—they follow from the mathematical theory (Theorems 4.5 to 4.7 and 5.1)—but the empirical verification provides a sanity check that the implementation correctly instantiates the theory.

9.3 Observations

1. **Concept count vs. state count.** For most benchmarks, the number of concepts is close to the number of states plus one (for the bottom concept with empty extent). This reflects

the fact that session type state spaces tend to have *distinct capability profiles*: each state enables a unique set of methods.

2. **Parallel composition creates sharing.** Protocols using the $\|$ constructor often have states that share enabled labels (e.g., both (s_1, s_2) and (s_1, s_2') may enable the same left-branch methods). This leads to larger concepts with non-singleton extents.

3. **Recursive protocols.** Recursive types create cycles in the state space, but the incidence relation is unaffected by cycles (it depends only on which transitions exist, not on reachability). Recursive protocols behave like their unfolded counterparts for concept analysis.

10 Lean 4 Mechanization

We mechanize the core results in Lean 4 using Mathlib [12], specifically the `Mathlib.Order.Rel.GaloisConnect` module which provides exactly the polarity-to-Galois-connection construction.

10.1 Mathlib Foundation

Mathlib defines `SetRel` as a binary relation between types and provides:

- `SetRel.leftDual` ($= -^{\uparrow}$): the label closure.
- `SetRel.rightDual` ($= -^{\downarrow}$): the state closure.
- `SetRel.gc_leftDual_rightDual`: the Galois connection between Set σ and $(\text{Set } \lambda)^{\text{op}}$.
- `SetRel.equivFixedPoints`: the bijection between left and right fixed points.

10.2 Our Contributions

Building on this foundation, we prove (file: `Reticulate/Polarity.lean`):

1. `state_closure_extensive`: $A \leq R.\text{rightDual}(R.\text{leftDual}(A))$.

2. `label_closure_extensive`: $B \leq R.\text{leftDual}(R.\text{rightDual}(B))$.

3. `state_closure_antitone`: $R.\text{rightDual}$ is antitone.

4. `label_closure_antitone`: $R.\text{leftDual}$ is antitone.

5. `state_closure_idempotent`: Double state closure is idempotent.

6. `label_closure_idempotent`: Double label closure is idempotent.

7. `fixed_point_bijection`: Fixed points are in bijection.

8. `concept_bot_le`: Bottom concept $\leq$ everything.

9. `concept_le_top`: Everything $\leq$ top concept.

All 9 results compile with **`zero sorry`**, verified by `lake build` (668 jobs, including all 18 Reticulate modules).

10.3 Proof Techniques

The key insight is that antitonicity + extensiveness suffices for idempotency. For `state_closure_idempotent`:

- ($\leq$): By extensiveness of label closure, $A^{\uparrow} \leq ((A^{\uparrow})^{\downarrow})^{\uparrow}$. By antitonicity of state closure, $(((A^{\uparrow})^{\downarrow})^{\uparrow})^{\downarrow} \leq (A^{\uparrow})^{\downarrow}$.
- ($\geq$): Direct from extensiveness of state closure applied to $(A^{\uparrow})^{\downarrow}$.

11 Related Work

Formal Concept Analysis. FCA was introduced by Wille [13] and systematized by Ganter and Wille [7]. It has found applications in data mining [11], information retrieval [10], software engineering (concept lattices for class hierarchies and feature models), and ontology construction. To our knowledge, this is the first application of FCA to session type state spaces.

Galois connections in programming languages. Cousot and Cousot's abstract interpretation framework [4] uses Galois connections between concrete and abstract domains. Our polarity Galois connection is different: it does not relate two levels of abstraction but rather two dimensions (states and labels) of the same structure. However, there may be interesting connections: one could view the concept lattice as an "abstract domain" for session type analysis.

Birkhoff's polarity. Birkhoff [1] introduced the polarity construction in §IV.9–10 of *Lattice Theory*. The general theory states that any binary relation between two sets induces a Galois connection whose fixed points form a complete lattice. Our contribution is the specific instantiation for session type state spaces and the empirical/mechanized verification.

Closure operators and lattice theory. The connection between closure operators and complete lattices is classical [9, 5, 8]. Erné et al. [6] provide a comprehensive survey of Galois connections in mathematics and computer science.

12 Conclusion

We have shown that every session type state space carries a natural formal context (states as objects, labels as attributes, enablement as incidence) whose concept lattice provides a second, independent lattice structure alongside the reachability lattice. The derivation operators form a Galois connection whose fixed points—the formal concepts—capture maximal capability clusters in the protocol.

All results are verified on 79 benchmark protocols (49 tests) and mechanized in Lean 4 with zero `sorry` (9 theorems/definitions). The implementation adds the `polarity` module to the Reticulate library, bringing the total to 2,189 passing tests across 20 modules.

Future work.

- **Relationship between the two lattices.** Is there a morphism (homomorphism, Galois connection) between the reachability lattice $\mathcal{L}(S)$ and the concept lattice $\mathfrak{B}(Q, \Sigma, R)$? Under what conditions are they isomorphic?
- **Attribute exploration.** FCA's attribute exploration algorithm [7] can discover *implications* ("if methods a and b are available, then c must also be available"). These are protocol invariants not easily derivable from the session type syntax alone.
- **Concept reduction.** Can concept lattice reduction (removing redundant concepts) simplify session type state spaces? This could be relevant for protocol minimization and testing.
- **Many-valued contexts.** Extending from binary (enabled/disabled) to many-valued attributes (e.g., distinguishing branch vs. selection transitions) could yield richer concept structures.

References

[1] Garrett Birkhoff. *Lattice Theory*, volume 25 of *Colloquium Publications*. American Mathematical Society, 3rd edition, 1967.

[2] Alexandre Zua Caldeira. The parallel constructor for session types: Specification. Technical report, 2025. https://github.com/alcides/reticulate.

[3] Alexandre Zua Caldeira. Session type state spaces form lattices. Preprint, 2025.

[4] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL 1977*, pages 238–252. ACM, 1977.

[5] Brian A. Davey and Hilary A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002.

[6] Marcel Erné, Jürgen Koslowski, Austin Melton, and George E. Strecker. A primer on Galois connections. *Annals of the New York Academy of Sciences*, 704(1):103–125, 1993.

[7] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

[8] George Grätzer. *Lattice Theory: Foundation*. Birkhäuser, 2011.

[9] Oystein Ore. Galois connexions. *Transactions of the American Mathematical Society*, 55(3):493–513, 1944.

[10] Uta Priss. Formal concept analysis in information science. *Annual Review of Information Science and Technology*, 40(1):521–543, 2006.

[11] Gerd Stumme, Rudolf Wille, and Uta Wille. Conceptual knowledge discovery in databases using formal concept analysis methods. In *Principles of Data Mining and Knowledge Discovery (PKDD 1998)*, volume 1510 of *LNCS*, pages 450–458. Springer, 1998.

[12] The Mathlib Community. The lean mathematical library, 2020. https://github.com/leanprover-community/mathlib4.

[13] Rudolf Wille. *Restructuring Lattice Theory: An Approach Based on Hierarchies of Concepts*, volume 83 of *NATO ASI Series*. Springer, 1982.

DRAFT